CODShield

The complete study guide

From "what even is this?" to "I can explain every file, table and arrow." Written for the team that built it and now wants to truly understand it.

AHA This guide is based on the project snapshot of 15 Oct 2026: a personal project using **Gemini** as the model. It separates what is **implemented** from what is only **planned**, because a table or event name in the code does not always mean the feature works.

How to read it

- **Part 1** explains everything in plain words, with a story. No code knowledge needed.
- **Part 2** is the big picture: the ecosystem diagram and the order-status map. Keep it open while reading the rest.
- **Part 3** is the deep dive: how each mechanism really works, to the technical detail.
- **Parts 4 to 6** map every file, every storage place, and one order line by line.
- **Part 7** is the one-liner revision sheet. **Part 8** is the glossary. **Part 9** lists the honest gaps. **Part 10** has run commands and a self-test.
- Look for the colored boxes: **AHA** (the key insight), **WATCH OUT** (a trap), **TRY IT** (do it yourself), **CHECK YOURSELF** (a question), **REMEMBER** (keep this).

The whole project in one breath

A customer chats with an online shop. An AI helper (Gemini) talks to them and proposes an order, but the **database** decides prices and stock. Every step is written as a small record (an **event**) in a PostgreSQL table. A separate program, the **worker**, reads those records one at a time and runs the right function. That chain cleans the address, scores the risk, phones doubtful customers, ships the parcel, and learns from refusals. A staff page lets humans step in. It is a prototype: the courier, the WhatsApp channel and most phone calls are simulated or unverified.

Part 1. Everything in plain words

Imagine you own a clothing shop in Lahore. Customers message you, you take orders, a courier delivers, and the customer pays cash at the door. The problem: some people refuse the parcel or never open the door. You already paid for the ad, the packing and the courier both ways. CODShield is a **team of robot and human staff** that protects the shop from this. Here is the cast.

Shop role	Real name in the code	What they do
The customer's window	`web/chat.html`	A WhatsApp-style page where the buyer types.
Front desk	`web_gateway.py, gateway.py`	Receives the message, writes it down, pins a slip on the notice board.
The notice board	the **events** table	A pile of slips, oldest first. Every important thing that happens becomes a slip.
The clerk	`worker.py`	Takes the oldest slip, hands it to the right staff member, and retries if they drop it.
The salesperson	`sales_agent` + Gemini	Chats with the customer and collects items, name and address. Cannot set prices.
The price book	the **products** table	The only truth about price and stock. The AI is not allowed to overrule it.
The tidy-up person	`address_clean.py`	Fixes messy addresses (DHA ph 5 lhr) and says how complete they are.
The detective	`risk.py`	Adds points for suspicious signs and writes down the reasons.
The manager	`orchestrator_*, set_status`	Moves the order to its next step. Only the manager changes status, and only by the rules.
The caller	`call_agent.py`	Phones doubtful customers: confirm, cancel or change address?
The courier	`simulate_courier.py`	Pretend for now. Says delivered or refused.
The shop staff	`web/staff.html`	Humans who take over a chat, edit products, and watch the overview.
The memory	the **risk_entities** table	Remembers phone numbers that refused twice.

Five golden ideas

- **1. The AI proposes, the database decides.** Gemini may suggest "2 shirts", but the app looks up the real price and stock, does the maths, and reads the total back to the customer.
- **2. Everything is a slip on a board.** Staff never shout at each other. They read slips and write new ones. That is the event queue.
- **3. Do it once, even if it is tried twice.** Every slip has a unique ticket number, so a retry cannot create a duplicate order or redial a customer.
- **4. Orders only move along allowed steps.** An order cannot jump from *created* to *delivered*. The rule book is in code.
- **5. Protect the people.** Phone numbers are scrambled for matching and locked for dialing. Secrets live in *.env*, never in Git.

AHA If you remember one thing: **the model talks, the database decides, the worker walks the order through its steps.**

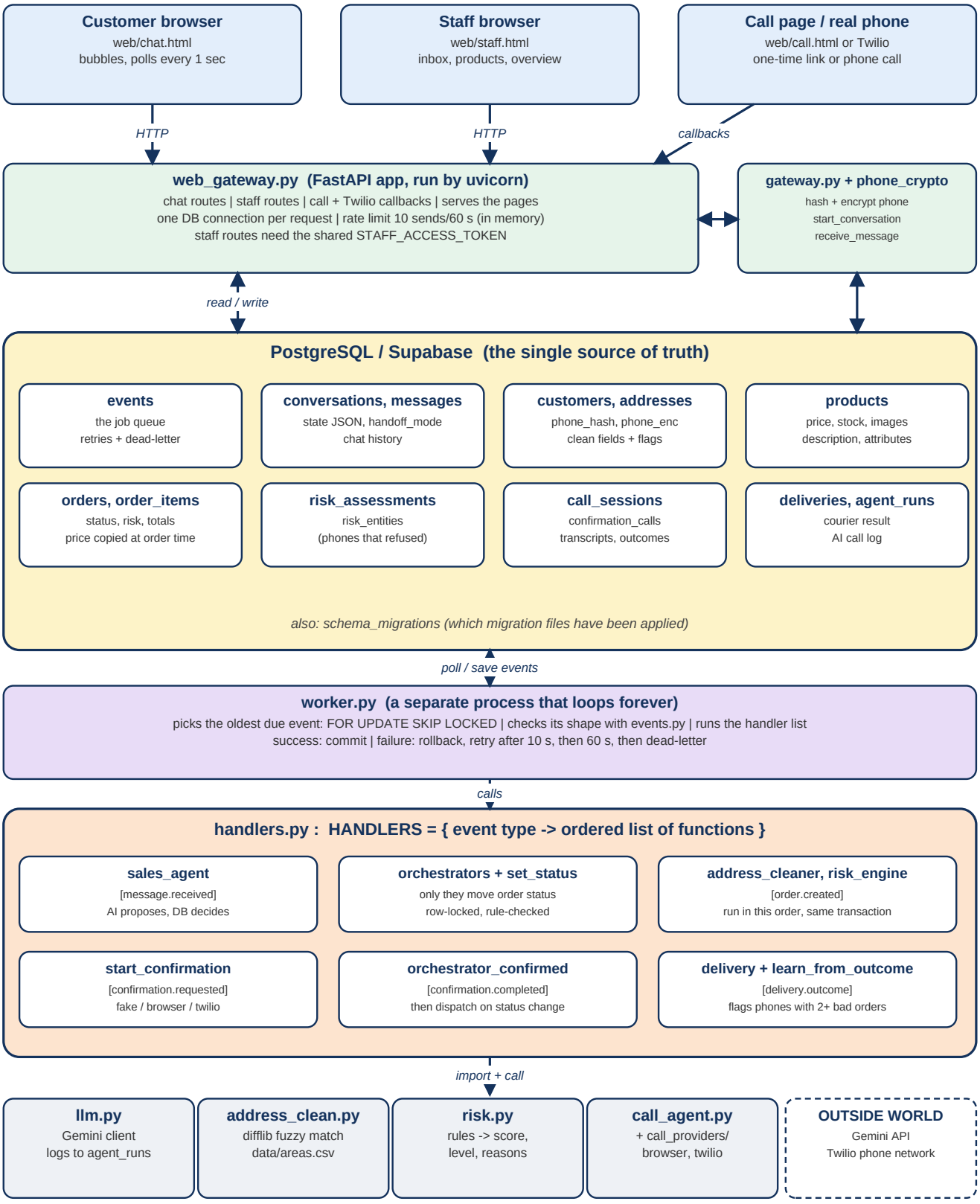
A day in the life of one order

Sara opens the chat page and types her phone number. The front desk hashes and locks it and opens a conversation. She writes "1 shirt chahiye". A slip *message.received* goes on the board. The clerk (worker) picks it up and gives it to the salesperson, who asks Gemini what to reply. Gemini suggests an order: 1 Cotton Shirt. The app finds the real shirt in the price book (Rs 1500, stock 100), and sends back: "1 Cotton Shirt, total Rs 1500, address?"

Sara gives "House 5, DHA Phase 5, Lahore" and says yes. Only now is the order created, one shirt leaves the stock, and an *order.created* slip appears. The manager moves it to *scoring*, the tidy-up person cleans the address (quality high), and the detective scores it low risk. Low risk means *cleared*, so it is dispatched. A pretend courier later says delivered. Had Sara typed just "DHA", the address score would be poor, the risk medium, and a confirmation call would have asked her first.

Part 2. The big picture

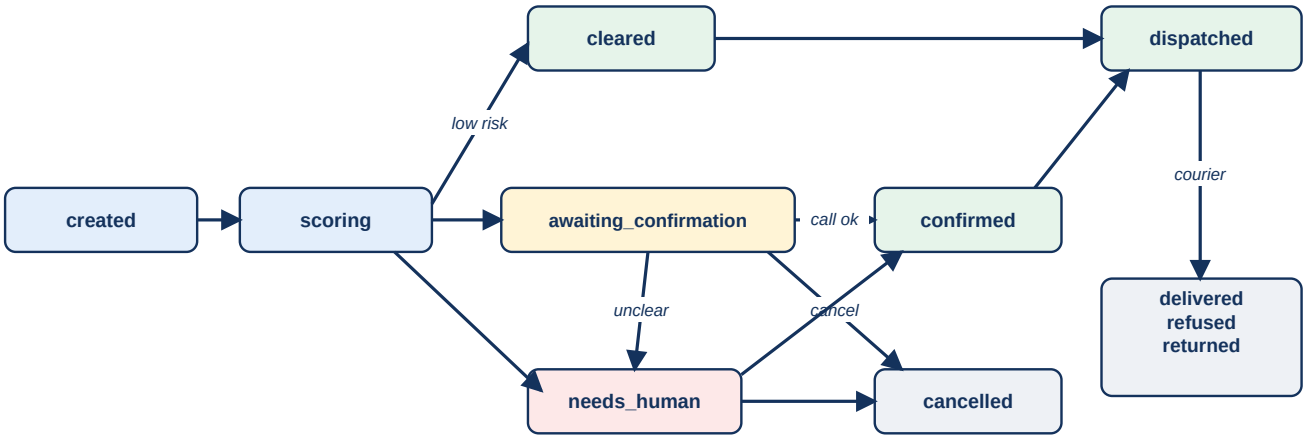
Ecosystem diagram. Read top to bottom: browsers talk to the web server, which writes to the database; the separate worker reads the database and runs handlers, which use helper modules and outside services.



Colors: blue = what people see, green = the web door, yellow = the database, purple = the worker, orange = the staff functions, grey = helper modules, dashed = outside your computer. Twilio calls back into web_gateway.py (signature checked). The command-line tools (fake_order.py, simulate_courier.py, smoke_test.py) write events straight into the events table.

The order-status map

Every arrow is one move that *events.can_transition* allows. **cancelled, delivered, refused, returned** are terminal: nothing follows. Anything not drawn here is blocked.



Status	Meaning
created	Order saved after the customer confirmed.
scoring	Address cleaning and risk scoring in progress.
cleared / awaiting_confirmation	Low risk goes straight on. Medium or high waits for a confirmation call.
needs_human	The call was unclear or unanswered. A person must decide. (Not the same as the chat handoff, see 3.9.)
confirmed / cancelled	The customer said yes or no (or changed the address, which also counts as confirmed).
dispatched	A demo delivery record exists (courier name *TCS (demo)*).
delivered / refused / returned	Final courier result. Refused and returned give the reserved stock back.

Part 3. How it really works (the deep dive)

3.1 The queue: the *events* table and the worker

A **queue** is a line of jobs waiting their turn. Big systems use Kafka or Redis. CODShield uses an ordinary PostgreSQL table instead, which is simpler and needs nothing extra to install.

events column	What it holds
id, type	Unique id, and the kind of slip, e.g. `message.received`.
seller_id, correlation_id	Which shop; and a grouping id (the conversation or the order) so you can follow one story across many events.
idempotency_key (UNIQUE)	The ticket number. Saving the same key twice does nothing.
payload (JSONB)	The whole event as JSON text the database can still search inside.
created_at, processed_at	When it was written; when it was finished. Empty processed_at means still waiting.
attempts, last_error, failed_at, next_attempt_at	Added by migration 004 for retries. last_error stores only the exception class name, never secrets.

- **Saving:** `db.save_event` runs INSERT ... ON CONFLICT (idempotency_key) DO NOTHING. A duplicate is silently ignored.
- **Claiming:** `worker.process_one` selects the oldest due, unfinished, not-failed event with **FOR UPDATE SKIP LOCKED**. FOR UPDATE locks the row so nobody else can take it. SKIP LOCKED means "if another worker holds it, just skip to the next". So you could run two workers safely.
- **Checking:** the JSON is validated by a Pydantic *TypeAdapter* over the Event union. If the shape is wrong, it fails here, before any handler runs.
- **Running:** every function in `HANDLERS[event.type]` runs in order, all inside **one database transaction** (all changes happen or none do).
- **Success:** mark processed and commit.
- **Failure:** rollback (undo everything that event did), attempts + 1, retry after **10 seconds**, then **60 seconds**; on the third failure failed_at and processed_at are set. That is a **dead letter**: parked and no longer retried. If a final failure was a customer message, the customer gets one generic technical-error reply.
- **Idle:** `run()` sleeps 1 second only when no event was processed.

WATCH OUT A handler that catches a Gemini error itself and writes "please try again" to the customer counts as **success**, so the event is not retried. Also, db.py still has an older `next_event` that ignores failed_at and next_attempt_at. The worker uses its own selector, so do not switch back.

REMEMBER One exception to "one transaction": confirmation startup commits a call-session marker *before* calling Twilio. If the worker then crashes and retries, the marker (backed by a unique index) stops a second phone call to the same person.

3.2 Events and contracts: *events.py*

Every event carries an **Envelope**: generated event_id, version, UTC occurred_at, seller_id, correlation_id and idempotency_key. The events are: message received, human handoff requested, staff takeover / reply / resume, order created, address normalized, order scored, confirmation requested, confirmation completed, order status changed, delivery outcome, entity flagged. They form a **discriminated union**: the field *type* tells Pydantic which class to build. A "contract" simply means everyone agrees on the exact fields. Ticket numbers look like `cc-{order_id}-{attempt}` for a call result, so call attempt 1 of an order can only be recorded once. *address.normalized* has no handler: it is an audit note that is saved and then marked done.

3.3 The status machine: *set_status*

Only the orchestrator functions should change status, through one function: it runs SELECT status ... **FOR UPDATE** (lock the order row), asks `events.can_transition(old, new)`, and if allowed updates the status and timestamp and

saves an *order.status_changed* event with an idempotent key. If not allowed it prints a blocked message and returns False. Two different ideas share the word "human": the order status *needs_human* (a call failed) and the conversation field *handoff_mode* = ai, pending or human (the chat needs staff).

WATCH OUT	`address_cleaner` also calls set_status(...'scoring'). That is a design slip, but harmless: orchestrator_created already moved the order to scoring, so the second call is blocked and does nothing.

3.4 The sales agent: where the AI stops and the database starts

- **Load context:** the seller, the whole product catalog (name, price, stock, description, attributes, image URLs), the conversation state and handoff mode, and the latest order. If staff have taken over (*handoff_mode = human*), the AI does not run at all.
- **Prompt:** the catalog is pasted into the system prompt. This is fine for 3 products and does not scale to hundreds. There is no tool-calling and no vector search yet, even though the schema has an embedding column.
- **Model reply:** Gemini must answer JSON like {reply, order}. It can propose an order; it cannot create one.
- `_prepare_order`**:** maps the product names to real product ids, **locks those product rows**, checks each quantity (max 20 per product) and stock, and computes the total with Python **Decimal** (exact money maths, no floating-point errors) using database prices.
- **Pending quote:** the result is saved as `pending_order` inside conversations.state (a JSON column), and the app sends its own read-back message, not the AI's wording.
- **Confirmation:** only a later message accepted by `_is_affirmative` (a yes-detector) confirms that stored quote. If prices changed since, a refreshed read-back is sent and the customer must say yes again. If stock is short, the customer is told what is available and no order is made. A cancel clears the pending cart.
- `create_order`**:** subtracts the reserved stock, creates the address, order and order_items (unit price copied in), stores the order id in conversation state, and saves *order.created*.
- **Handoff:** if the customer asks about product information that is not in the approved catalog, the app records *human.handoff_requested* and sets handoff_mode to *pending*. The AI keeps working while pending, and sends one handoff notice (not one per reply). After takeover it goes silent until staff resume it.

CHECK YOURSELF	Why is the order total computed in the app and not by Gemini? *(Because a model can be wrong, or talked into a fake price. Money must come from the database.)*

3.5 The model client: *llm.py*

It calls Gemini through the **OpenAI Python SDK** pointed at Google's **OpenAI-compatible endpoint** (same code shape, different address). The client is created once and cached. It uses `httpx.Client(trust_env=False)` so a weird shell proxy setting cannot hijack the request. Defaults from the notes: the Flash model name set in the file, `LLM_MAX_TOKENS` 1500, a 15-second timeout, the SDK retrying once, and low reasoning effort (unless the model name contains flash-lite). `ask_json` asks for JSON; if the first answer is not the required shape it asks once more (so up to two completions). It deletes any <think> blocks, strips Markdown code fences, parses the outermost { ... }, and returns the result plus token counts. Every attempt is written to **agent_runs**.

WATCH OUT	llm.py hardcodes *agent = 'sales'* and never records the order id. So call-agent usage is mislabeled, and the staff overview's per-order cost usually shows nothing. Setting a token price will not fix that.

3.6 The address cleaner: *address_clean.py*

A **pure** function (it does not touch the database): give it raw text, get structure back. It loads `data/areas.csv` once at import (columns city, area, zone, type, aliases; only city, area and aliases are used). Steps: fold case and normalize text, expand abbreviations (ph, blk, lhr...), look up city and area aliases, **fuzzy-match** words with the standard library's *difflib* so "Jhohar Town" still finds "Johar Town", then pull out house number, block/phase/sector, street and landmark. It sets **flags** (for example missing_house_number, missing_city, too_short, landmark_only), computes a transparent 0 to 1 **quality score**, builds a tidy display string, and creates an **address hash** from the canonical text `house|block|area|city` plus HASH_SALT. Street and landmark are *not* in the hash. The handler stores city, area,

street, quality_score, addr_hash and the flags (JSONB) on the addresses row. No maps, no coordinates.

WATCH OUT	Two incomplete addresses with no house, block or area can share the same canonical hash, so a future fraud graph could link unrelated people. Also, areas.csv alias quality has not been audited.

3.7 Risk scoring: *risk.py*

Plain rules, no machine learning. Start at 0.04, then add: order amount (up to 0.12, scaling between Rs 5,000 and Rs 30,000), first-time customer 0.06, late-night order 0.04, many recent orders (capped at 0.14), flagged phone 0.24, plus smaller additions for refusal history, linked identities, poor address quality and address flags. The total is capped at 0.999. **Levels:** below 0.35 is *low*, 0.35 up to 0.75 is *medium*, 0.75 and above is *high*. The result, with reasons in plain English and the input features, is saved in **risk_assessments** and copied to the order.

WATCH OUT	At runtime `risk_engine` only supplies: first-time, recent-orders, phone-flagged, address quality and flags. It does not look up detailed refusal history or linked identities, so those richer rules in risk.py are not used by the live flow. The legacy `score_order` call shape is kept so older tests still pass.

3.8 Confirmation calls

- **Provider switch:** `CALL_PROVIDER` = fake (default), browser, or twilio. Fake records a random outcome unless FAKE_CALL_OUTCOME is set.
- **Start:** `start_confirmation` creates a **call_sessions** row (provider, status, a *hash* of a one-time token, transcript JSON, times) and commits it. A **partial unique index** on (order_id, attempt) for browser and Twilio sessions means a retry finds the old session instead of redialing. Real calls only happen between 09:00 and 21:00 Pakistan time.
- **Browser provider:** prints a one-time URL (`/call/{token}`). web/call.html reads the token, speaks the opening line with SpeechSynthesis, listens with SpeechRecognition (ur-PK) if the browser has it, and always keeps a text box as fallback.
- **Twilio provider:** reads the *encrypted* phone, decrypts it with PHONE_ENC_KEY, and sends an HTTP POST to Twilio's Calls API using Python's urllib with a 10-second timeout. If the number is missing or unreadable, or (in trial mode) not in TWILIO_VERIFIED_NUMBERS, it does not dial and records why. Twilio then calls back into web_gateway; each callback is checked with an **HMAC-SHA1 signature**, and the reply is **TwiML** (Twilio's XML) with a Gather that accepts speech (ur-PK) and keypad (**DTMF**).
- **The dialogue:** `call_agent.next_turn` reads order facts from the database, and the spoken lines are fixed app text, not model output, so the model cannot promise discounts or change prices. Exact yes/no/change-address phrases and keypad 1 or 2 are decided by plain code. Gemini is used only for ambiguous replies and may only answer "changed address". Two silences means *no_answer*. One speech-recognition failure retries, a second offers keypad or text. Six customer turns means *unclear*.
- **Finish:** `finish_call` locks the session, writes confirmation_calls (outcome, transcript, duration), completes the session, and saves *confirmation.completed* with key cc-{order}-{attempt}.
- **Outcome mapping:** confirmed or changed_address -> *confirmed*; cancelled -> *cancelled* (stock restored once); no_answer or unclear -> *needs_human*. A changed address adds a new cleaned address row and repoints the order; the old row stays, and risk is not recalculated.

WATCH OUT	The Twilio HTTP request happens inside the worker. For up to 10 seconds the worker may be unable to process other events. And a changed address stores the customer's *whole sentence* as the new raw address; no clever extraction yet.

3.9 Staff side: handoff, products, overview

- **Auth:** one shared *STAFF_ACCESS_TOKEN* sent as a header. The page keeps it in sessionStorage. There are no accounts or roles.
- **Inbox:** the page polls every 2 seconds. Staff pick a pending conversation, **take over**, **reply**, or **resume AI**. Each action is queued as an event and handled by the worker, same as everything else.

- **Products:** create or edit name, price, stock, description, attributes and images (JPG, PNG, WebP). Pictures are saved on the server's local disk in `uploads/products/`.
- **Overview (cockpit):** polls every 10 seconds: order counts, delivered value, recent risk, call and delivery results, recent events and a deterministic daily briefing (plain code, not AI). "Delivered value" is sales, not profit and not money saved.

WATCH OUT	No notification exists: staff only see a handoff if the page is open. And several staff routes look up records by id without always limiting them to one seller, so this is not safe for multiple shops yet.

3.10 Stock and the learning loop

Stock: reserved (subtracted) when the order is created, restored exactly once if the order is cancelled, refused or returned, and only if that status change actually succeeded. **Learning:** when a delivery outcome arrives, `learn_from_outcome` updates the deliveries row and the phone's row in **risk_entities** (counts of total, refused, returned orders). After 2 or more refused or returned orders that phone is flagged and an *entity.flagged* event is saved. The next order from that phone gets +0.24 in risk. Only phones are learned; address and device hashes and the entity_links graph table are not used yet.

3.11 Privacy and security in one place

- **Hash** (SHA-256 of HASH_SALT + digits): a one-way fingerprint for matching phones and addresses. **Encrypt** (Fernet with PHONE_ENC_KEY): a two-way lock so a call can dial. Both are needed.
- Phones are normalized first (0300..., 300..., 0092... all become +92300...). Without this the same person would hash differently.
- If HASH_SALT is unset a demo salt is used. Change it later and old hashes no longer match new ones. Lose PHONE_ENC_KEY and stored phones can never be decrypted.
- Secrets live in .env (ignored by Git). SQL always uses %s parameters. The send limiter lives in memory, so it resets on restart and is not shared between server processes. Pages escape and allowlist image URLs.

Part 4. File by file: who calls whom

"Calls" means imports or invokes. "Touches" means the tables or storage it reads or writes. Remember: the business flow is event-driven, but the code does have direct imports. Both are true.

File	Job	Calls / imports	Touches
`web_gateway.py`	HTTP door: chat, staff, call, Twilio routes; serves pages; input validation	gateway.py, call_agent, db	customers, conversations, messages, events, products, orders; uploads/products/
`gateway.py`	Front desk: hash phone, start chat, save message + event	phone_crypto, events, db	customers, conversations, messages, events
`phone_crypto.py`	Normalize PK phones; Fernet lock/unlock	cryptography	reads PHONE_ENC_KEY
`worker.py`	Event loop with retry and dead-letter	db, events, handlers	events
`handlers.py`	All business logic; the HANDLERS map	db, llm, risk, address_clean, call_agent, providers, events	almost every table
`events.py`	Event models, status map, can_transition	pydantic only	nothing (definitions)
`db.py`	Connect, save_event, mark_done	psycopg2, dotenv	events
`llm.py`	Gemini client; ask_json with retry	openai SDK, httpx	agent_runs; Gemini over internet
`risk.py`	Rule scoring	address_clean (assess_order only)	nothing (pure)
`address_clean.py`	Clean address + hash	difflib, csv	reads data/areas.csv
`call_agent.py`	Call dialogue, finish_call	db, llm	call_sessions, confirmation_calls, events
`call_providers/browser.py`	Prints one-time call link	none	nothing
`call_providers/twilio.py`	Twilio REST call, signature check, TwiML	phone_crypto, urllib	customers.phone_enc, call_sessions
`migrate.py`	Applies migrations once each	db	schema_migrations
`seed.py`	Demo Store + 3 products	db	sellers, products

File	Job	Notes
`web/chat.html`	Customer chat page	sessionStorage holds the conversation id; polls every 1 s; typing indicator; 60 s slow-reply notice; image URL allowlist; ?debug=1 shows risk.
`web/staff.html`	Staff page	Token in sessionStorage; inbox, products, overview; image preview.
`web/call.html`	Browser call	Speech tools when available, text box always.
`fake_order.py`	Creates an order with no AI	Writes customer, address, order, items and order.created; the worker must process it. Leaves rows behind.
`chat_cli.py`	Terminal chat	Needs worker, DB, PHONE_ENC_KEY, Gemini. Waits up to 90 s for a reply.
`simulate_courier.py`	Pretend courier	Refusal chance: low 0.08, medium 0.25, high 0.50.
`smoke_test.py`	Two real orders through the pipeline	Run on a demo database with the worker stopped, or they race.
`test.py`	Tiny diagnostic	Prints one event and two transition checks. Not a test suite.
`scripts/eval_sales.py`	Paid model exam	15 cases, at least 30 Gemini calls. Run only on purpose.
`migrations/002..005`	Database changes	002 call_sessions; 003 address_flags; 004 event retries and unique call index; 005 product info, media_urls, handoff fields.
`tests/*`	Offline tests	unittest files cover address, sales flow, worker, calls, providers, phone crypto, web gateway (45 passed at last run). test_sales_agent.py uses pytest style and is skipped by unittest.
`requirements.txt`	Packages	Unpinned. streamlit is listed but there is no dashboard. pytest is missing.

File	Job	Notes
`.env.example` / `.gitignore`	Config names / Git rules	Names only, no values. .env, keys, uploads and caches are ignored.
`AGENTS.md, README.md`	Docs	Both are partly stale. Trust the code and this guide.

Part 5. What is stored where

When something "disappears", this table tells you where to look.

What	Where it lives	Details
Secrets	`.env` on your computer	Database link, Gemini key, PHONE_ENC_KEY, HASH_SALT, staff token, Twilio settings. Git ignores it.
Chat history	messages	direction in/out, kind text/voice, body, media_urls.
Conversation memory	conversations.state (JSON)	pending_order and order_id. Also handoff_mode (ai, pending, human), reason, request time.
Customers	customers	phone_hash (unique per seller), phone_enc (locked), name.
Addresses	addresses	raw_text, city, area, street, quality_score, addr_hash, address_flags. lat/lng stay empty.
Catalog	products	sku, name, Urdu name, price, stock, description, attributes JSON, image_urls. Embedding column exists but is unused.
Product pictures	disk: uploads/products/	Not in Git, not in the database. Lost if the host disk is wiped.
Orders	orders, order_items	Status, total, risk fields. unit_price is copied at order time, so later price edits do not change old orders.
Stock	products.stock	Subtracted at order creation, restored once on cancel, refused or returned.
Risk reasons	risk_assessments	score, level, reasons, features. Also copied to orders.risk_score and risk_level.
Bad-phone memory	risk_entities	Per phone hash: total, refused, returned, flagged.
Calls	call_sessions, confirmation_calls	Sessions hold token hash, provider, status, transcript. Calls hold outcome, transcript, duration.
Deliveries	deliveries	Courier (demo name), outcome, reason.
The queue	events	Every event and its retry state.
AI log	agent_runs	Model, tokens, latency. No order id.
Migration ledger	schema_migrations	Names of applied migration files.
In the browser	sessionStorage	Chat conversation id; staff token. Cleared when the tab closes.
In worker / server memory	Python process	Rate-limit counters and the cached Gemini client. Lost on restart.
Areas list	data/areas.csv	Read once when address_clean is imported.
Leaves your machine	Gemini, Twilio	Gemini sees chat text and the catalog. Twilio receives the phone number it dials.

Part 6. One order, line by line

The full technical journey. "Emits" is the event slip written at the end of that step.

#	Trigger	Function	Writes	Emits
1	POST /api/chat/start	gateway.start_conversation	customers (hash + locked phone), conversations	-
2	POST /api/chat/send	gateway.receive_message	messages (inbound)	message.received
3	worker loop	process_one	locks the event row	-
4	message.received	sales_agent -> llm.ask_json	messages (outbound), state.pending_order	-
5	customer says yes	_prepare_order, create_order	products.stock, addresses, orders, order_items, state.order_id	order.created
6	order.created	orchestrator_created	orders.status = scoring	order.status_changed
7	order.created	address_cleaner	address fields, quality, hash, flags	address.normalized
8	order.created	risk_engine	risk_assessments, orders.risk_*	order.scored
9	order.scored	orchestrator_scored	status cleared or awaiting_confirmation	order.status_changed, maybe confirmation.requested
10	confirmation.requested	start_confirmation	call_sessions, provider action	later confirmation.completed
11	call page or Twilio callback	call_agent.next_turn, finish_call	call_sessions, confirmation_calls	confirmation.completed
12	confirmation.completed	orchestrator_confirmed	status; stock back if cancelled; new address row if changed	order.status_changed
13	status cleared or confirmed	orchestrator_dispatch	deliveries (TCS demo), status dispatched	order.status_changed
14	you run the script	simulate_courier.py	-	delivery.outcome
15	delivery.outcome	orchestrator_delivery_outcome	status final; stock back if refused or returned	order.status_changed
16	delivery.outcome	learn_from_outcome	deliveries, risk_entities	entity.flagged (at 2+ bad)

TRY IT Run *python fake_order.py* twice, once pressing Enter and once typing DHA, with the worker running. Compare the risk reasons printed. Then open /staff and find both orders in the overview.

Part 7. One-liners: the revision sheet

Files

- `web_gateway.py`: the FastAPI web server and all HTTP routes.
- `gateway.py`: turns a customer message into a database row plus an event.
- `phone_crypto.py`: cleans phone numbers and locks them with Fernet.
- `worker.py`: loops forever, claims one event, runs handlers, retries on failure.
- `handlers.py`: all the business logic and the event-to-function map.
- `events.py`: the shape of every event and the legal status moves.
- `db.py`: opens connections and saves events safely.
- `llm.py`: the Gemini client that insists on JSON and logs usage.
- `risk.py`: adds up suspicious signs into a score, a level and reasons.
- `address_clean.py`: fixes messy Pakistani addresses and makes a hash.
- `call_agent.py`: the logic of one confirmation call.
- `call_providers/browser.py`: makes a one-time link for a demo call.
- `call_providers/twilio.py`: real phone dialing, signature checks and TwiML.
- `migrate.py`: applies each SQL change once and remembers it.
- `seed.py`: creates Demo Store with three products.
- `web/chat.html`: the customer's chat window.
- `web/staff.html`: the staff inbox, product editor and overview.
- `web/call.html`: the browser-based call page.
- `fake_order.py`: an order without chat or AI, for testing.
- `simulate_courier.py`: invents delivery results.
- `smoke_test.py`: runs two full orders as a health check.
- `scripts/eval_sales.py`: a paid exam for the sales agent.
- `data/areas.csv`: the list of known cities and areas.
- `migrations/`: numbered SQL files that change the database.

Ideas

- **Event:** a slip saying something happened. **Queue:** events waiting their turn. **Worker:** the program that works through them.
- **Idempotency key:** a ticket number so the same thing is never done twice.
- **Transaction:** a bundle of database changes that all succeed or all vanish.
- **Row lock (FOR UPDATE):** "I am using this row, wait." **SKIP LOCKED:** "if it is busy, take the next one."
- **Dead letter:** an event that failed three times and was parked.
- **State machine:** a rule book of allowed status moves.
- **Hash:** a one-way fingerprint. **Encryption:** a two-way lock with a key.
- **Salt:** a secret added before hashing so fingerprints cannot be guessed from a public list.
- **Webhook:** another service calling your web address. **Signature:** proof the call really came from Twilio.
- **AI proposes, database decides:** money and stock never come from the model.
- **Pending order:** a stored quote waiting for the customer's yes.
- **Handoff mode:** ai, pending or human, for the conversation. **needs_human:** an order status. Different things.

Numbers worth memorizing

- Message limit 500 characters. Quantity limit 20 per product. Send limit 10 per 60 seconds per conversation.
- Chat polls every 1 s; staff inbox every 2 s; overview every 10 s; slow-reply notice after 60 s.
- Event retries after 10 s, then 60 s, dead letter on the 3rd failure.
- Risk: low below 0.35, medium 0.35 to below 0.75, high 0.75 and up. Base 0.04. Flagged phone +0.24. First-time +0.06. Late night +0.04.
- A phone is flagged after 2 or more refused or returned orders.
- Calls allowed 09:00 to 21:00 Pakistan time. Twilio request timeout 10 s. Silence twice = no_answer. Six turns = unclear.
- Fake courier refusal: low 0.08, medium 0.25, high 0.50.
- Gemini timeout 15 s, max tokens 1500, up to two completions per question.
- Tests: 45 unittest tests passed at the last run, plus 8 pytest-style functions run separately.

Part 8. Glossary of the hard words

Term	Plain meaning
API / endpoint	A set of web addresses a program can call. One address (like /api/chat/send) is an endpoint.
FastAPI / uvicorn	FastAPI is the Python toolkit that defines the endpoints. uvicorn is the program that actually runs it and listens for visitors.
Process	One running program. The web server and the worker are two separate processes.
Polling	Asking again and again ("any news?") instead of being told. The chat page polls every second.
JSON / JSONB	A text format for structured data. JSONB is Postgres's searchable version.
Payload	The actual data inside a message or event.
Transaction, commit, rollback	A bundle of changes. Commit saves them all; rollback cancels them all.
Row lock	A claim on one database row so nobody else changes it at the same moment.
Idempotent	Safe to do twice: the second time changes nothing.
Retry / backoff	Trying again after a wait that grows (10 s, then 60 s).
Dead letter	A job that keeps failing and is set aside for a human to inspect.
Schema / migration	The blueprint of tables; a numbered file that changes that blueprint.
Enum	A column that can only hold values from a fixed list, such as order status.
Foreign key	A column pointing at a row in another table (an order points at its customer).
Partial unique index	A uniqueness rule that applies only to some rows, used so one call session exists per order attempt.
pgvector / embedding	A Postgres add-on that stores meaning-numbers for text so you can search by similarity. Present in the schema, unused by the app.
Pydantic	A library that checks data has the right shape and turns JSON into Python objects.
Discriminated union	A choice between several shapes decided by one field (here: type).
Hash / SHA-256 / salt	A one-way scramble; the specific scramble used; a secret ingredient added before scrambling.
Fernet	A ready-made, safe way to lock and unlock text with a single key.
HMAC-SHA1	A way to prove a message was signed with a shared secret. Twilio uses it on every callback.
Webhook	When another service calls your web address to tell you something.
TwiML	Twilio's XML instructions telling a call what to say and listen for.
DTMF	The tones from pressing phone keys. Pressing 1 to confirm is DTMF.
Gather	A TwiML instruction that listens for speech or keypad input.
Speech recognition / synthesis	Turning speech into text; turning text into speech. Browsers offer both for free.
LLM / prompt / token	A large language model; the instructions you send it; a chunk of text it counts and charges by.
OpenAI-compatible endpoint	Gemini offering the same request format as OpenAI so one SDK works for both.
Decimal	Python's exact number type for money. Ordinary floats can give 0.1 + 0.2 = 0.30000000000000004.
Fuzzy matching	Finding the closest spelling instead of an exact one.
Canonical	The one standard form of something, used so different spellings compare equal.
Mock / monkeypatch	A fake stand-in used in tests so no real database, Gemini or Twilio is touched.
sessionStorage	Browser memory that lasts only until the tab closes.
Rate limit	A cap on how fast one client may send requests.
Multi-tenant	One system serving many separate shops safely. This project is not hardened for that yet.
Demo vs production	Demo works on your laptop for showing; production must survive real users, attacks and crashes.

Part 9. Honest gaps and caveats

Do not describe a feature as working just because a table, event or README mentions it. These are the gaps that matter.

Gap	Why it matters	Idea to fix
MAX_CONVERSATION_MESSAGES = 40 exists but is never enforced	A long chat is not capped, so cost can grow.	Enforce it in sales_agent.
Whole catalog sits in the Gemini prompt	Fine for 3 products, slow and costly for hundreds.	Tool-calling plus search.
No base schema file in the repo	A fresh database cannot be rebuilt. Migrations need the core tables first.	Restore the core schema as a file.
Demo salt fallback for hashes	Production hashes would be guessable and fragile.	Set a private, stable HASH_SALT now.
Order-linked AI usage is empty	Per-order cost in the overview is mostly blank.	Pass order_id and the right agent name to ask_json.
"Rs saved" is not calculated	Delivered value is not profit or savings.	Build a replay or backtest.
No voice-note path	voice events exist but nothing transcribes.	Add speech-to-text later.
Twilio and Urdu voice unverified	Real calls may fail or sound wrong. Say uses en-IN.	Test on a trial account with a verified number.
Twilio call blocks the worker for up to 10 s	Other events wait.	Move calls to a queue or thread.
Changed address saves the whole sentence	Messy or wrong address; risk not re-run.	Extract and re-score.
Risk learning is phone-only	Address rings and linked phones are not caught.	Build the fraud graph on entity_links.
Uploads on local disk	Pictures vanish if the host is wiped.	Object storage.
Single shared staff token; routes not always seller-scoped	Unsafe for more than one shop or the open internet.	Real accounts and seller checks.
Rate limit in memory	Resets on restart; not shared across processes.	Store counters in the database or Redis.
No Dockerfile, CI, LICENSE, WhatsApp or courier connector	Hard to deploy or share; the channels are demos.	Add them when needed.

WATCH OUT **Git state at the last check:** the app source files were untracked locally and .gitignore was modified; only the README was committed. Run *git status*, then commit and push so your work is backed up on GitHub. Also, the README and AGENTS.md still contain stale statements (for example hackathon and Nebius mentions, a missing db/schema.sql, a missing LICENSE), so rewrite them to match the Gemini personal-project reality.

REMEMBER Safe claims: the offline logic is tested with mocks (45 unittest tests). Not verified: live Supabase behaviour, real Gemini answers, real browser speech, Twilio, public hosting.

Part 10. Run it, then test yourself

Commands (PowerShell)

Step	Command
Install	`python -m pip install -r requirements.txt` (and `pip install pytest` if you want the pytest-style tests)
Configure	`Copy-Item .env.example .env`, then fill DATABASE_URL, GEMINI_API_KEY, PHONE_ENC_KEY, HASH_SALT, STAFF_ACCESS_TOKEN. Make a lock key with `python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`
Database	Core tables must already exist. Then `python migrate.py` and `python seed.py`
Run	Window 1: `python worker.py` Window 2: `python -m uvicorn web_gateway:app --reload`
Open	http://127.0.0.1:8000/ for the customer chat, and /staff for the staff page
Helpers	`python fake_order.py`, `python simulate_courier.py`, `python smoke_test.py` (stop the worker first)
Tests	`python -m unittest` (offline, mocked). The paid exam: `python scripts/eval_sales.py` only on purpose.

Self-test: 12 questions

Q1. Why does the app compute the total instead of trusting Gemini?

Answer: Money and stock must come from the database. A model can err or be tricked.

Q2. What does ON CONFLICT DO NOTHING on idempotency_key protect against?

Answer: Saving the same event twice, for example on a retry.

Q3. What does SKIP LOCKED let you do?

Answer: Run several workers without two taking the same event.

Q4. Name the three retry outcomes of a failing event.

Answer: Retry after 10 s, retry after 60 s, then dead letter on the third failure.

Q5. Why hash phones AND encrypt them?

Answer: Hash for matching across orders; encryption so a real call can dial.

Q6. What happens if HASH_SALT changes later?

Answer: New hashes will not match old ones, so repeat-customer matching breaks.

Q7. Which status does a no-answer call lead to?

Answer: needs_human.

Q8. When is stock given back?

Answer: On cancel, refused or returned, once, and only if the status change succeeded.

Q9. Why is the Twilio redial guarded?

Answer: A committed call-session marker with a partial unique index stops a retry from calling twice.

Q10. What does a risk score of 0.50 mean?

Answer: Medium (0.35 to below 0.75): the order gets a confirmation call.

Q11. Is the fraud graph live?

Answer: No. Only phone-level flagging after 2 or more bad outcomes works.

Q12. Which parts are verified only by mocks?

Answer: Almost all external behaviour: Gemini, Supabase, browser speech, Twilio.

AHA	If you can answer all twelve without looking, and explain the ecosystem diagram out loud to a friend, you understand CODShield.